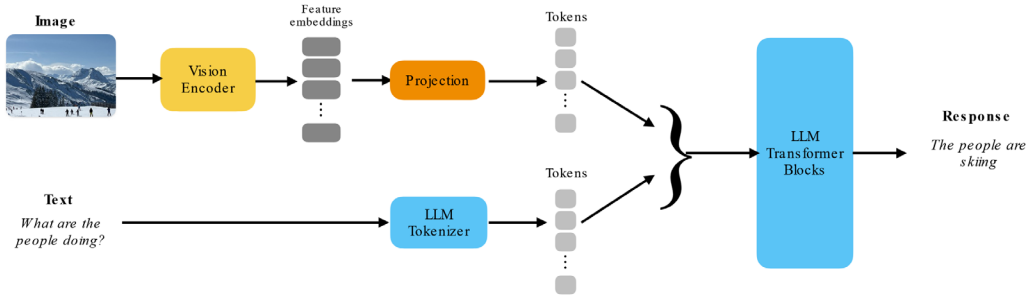
VISION-LANGUAGE MODELS

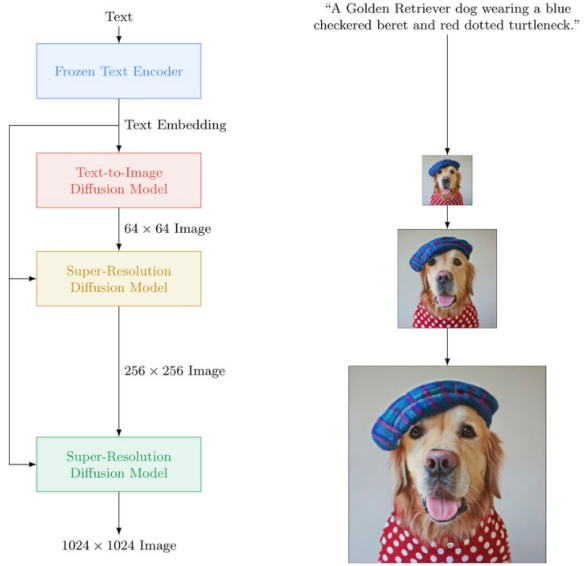
Lecture 1

Introduction and Vision Transformers

Why vision + language?

- **Models for analysis of image and text:**
  - Images: CNNs and image models — perception: what is in this image?
  - Text: Transformers / LLMs (GPT etc) — language: describe, reason, answer, instruction following
- **Many useful tasks require both language and images/videos:**
  - "What is the man in the red shirt holding?" — question answering based on image pixels
  - "Read this receipt and total the amounts" — OCR and reasoning
  - "Find the moment the goal is scored" — temporal analysis of images in video
  - "A watercolor fox in the snow" — text as the interface to image generation
- Vision-Language Models (VLMs) are core part of image/video to text
- A subset of Mutimodal Machine Learning Models (video + audio + …)
- Will also look at generative models (text to image mainly)





Module Overview

- **Prerequisites (assumed known)**
  - Transformers: encoder/decoder, self-attention, embeddings; GPT-style LLMs
  - CNNs; core ML; standard (non-variational) autoencoders
- **New architectures introduced in this course**
  - Vision Transformers
  - VAEs, GANs, diffusion models (mainly for image generation)
  - Other architectures build on these core models
  - Focus is more on concepts than low level mathematical formalism
- **Assessment**
  - Lab 1 + Lab 2 notebooks (done in Senior Lab and submitted)
  - Two-week (open) prac: build/evaluate a simple vision-language understanding application
- **Learning resources**
  - Bishop (see Notes to Students) as a reference – very technical/stats heavy (covers/touches on the core architectures above)
  - Online and paper references for those who want more detail (see last slide)

Outline: teaching a transformer to see

I

From words to patches

How an image becomes a token sequence: patchify → flatten → project. The [CLS] token. Full ViT architecture

II

Positional encoding for image tokens

Why 1D order is not enough in 2D. Learned vs sinusoidal; what learned embeddings discover; changing resolution by interpolation.

III

ViT vs CNN

Inductive biases; why ViTs need scale (how DeiT fixed it); the model zoo and the patch-size tradeoff; ViT as universal backbone.

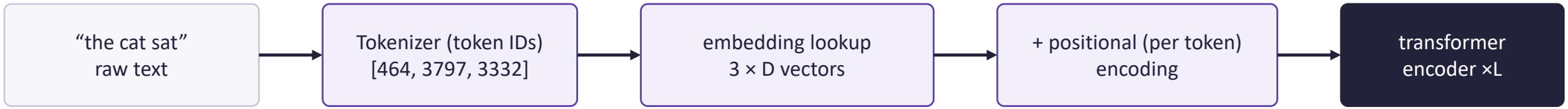
Question: what must change to move a transformer from text to images — and what can stay the same?

PART I

From words to patches

The input pipeline: image → tokens → embeddings — analogous to text processing

Recap: the text pipeline



- **The transformer core is modality-agnostic:**
  - Self-attention only sees “a set of D-dimensional vectors with positional information”
  - Nothing in $QK^T/\sqrt{d}$, softmax, MLP blocks, LayerNorm, or residuals mentions “text”

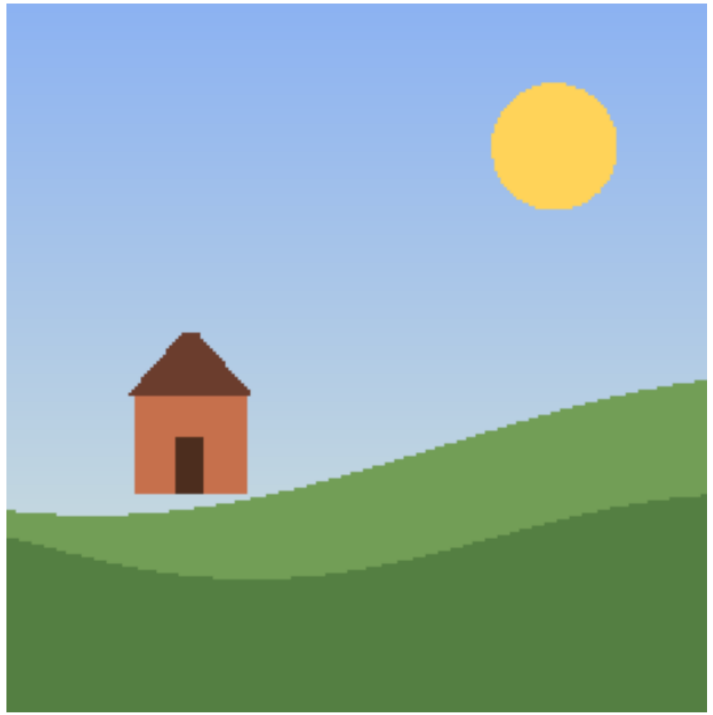
- **Thus, to process images, we “only” need to replace the front of the pipeline**
  - Question 1 — what plays the role of a token? (Part I)
  - Question 2 — what plays the role of position? (Part II)

Question: Why not just attend over pixels?

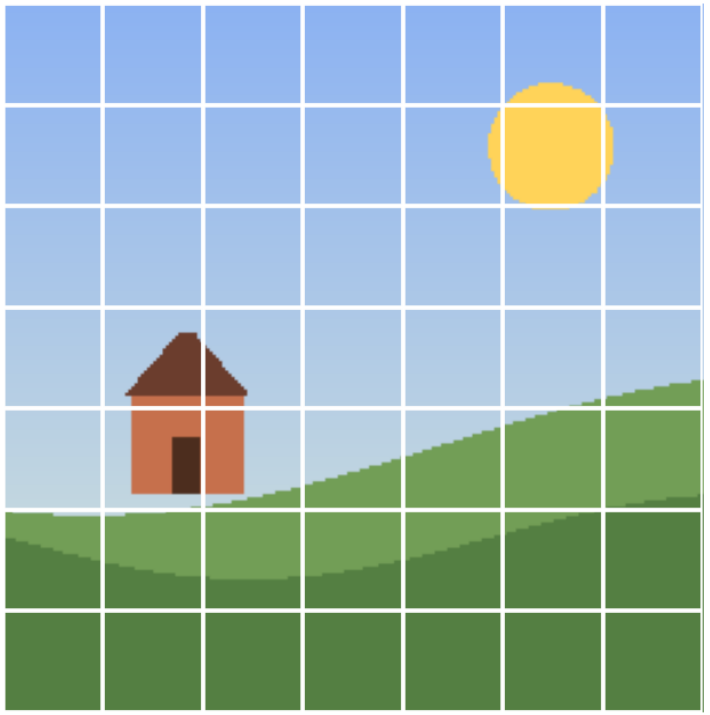
- **Simplest idea: one pixel = one token**
  - A modest 224 × 224 RGB image → 50,176 pixel (spatial) positions
  - Self-attention cost grows as $O(n^2)$ in sequence length n
  - $50{,}176^2 \approx 2.5 \times 10^9$ attention scores per head, per transformer layer vs $\approx 2.6 \times 10^5$ for a 512-token sentence
  - ≈ 25,000× more expensive than the text case — and what about memory for weights?
- **Also: a single pixel is a meaningless "word"**
  - One RGB triple carries almost no meaning on its own — unlike a word
  - Meaning in images exists at the level of local groups of pixels: edges, textures, parts
  - Colour may not even be needed for sensible structure e.g. grey-scale/single channel images, but need local structure
- **We need a coarser token — big enough to mean something, cheap enough to compute**
- **The ViT idea: use 16x16 pixel patches as tokens**
- **So: 50,176s token goes down to only 196 tokens for example above….**

The ViT idea: “an image is worth 16×16 words”

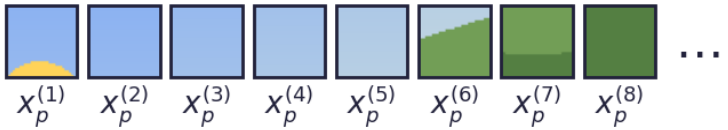
Input image $x \in \mathbb{R}^{224 \times 224 \times 3}$



Split into $P \times P$ patches
(here $P = 32 \rightarrow 7 \times 7 = 49$ patches)

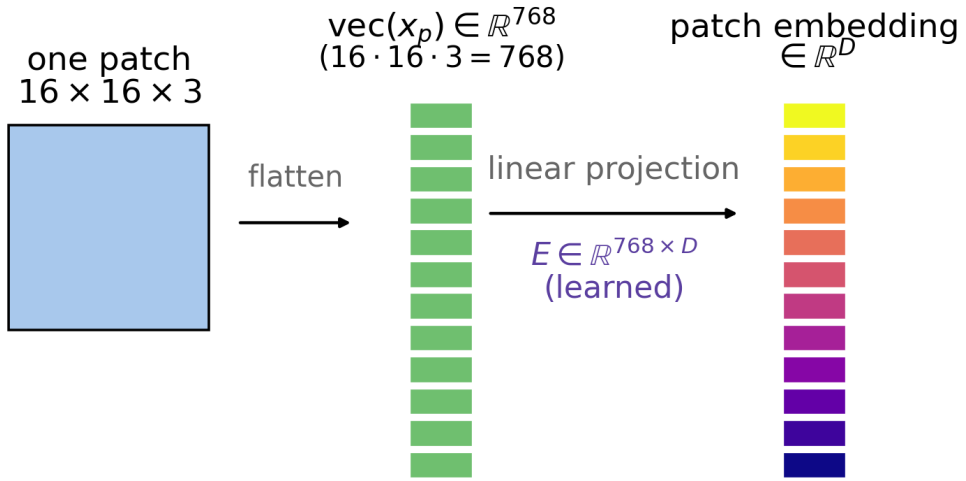


Flatten grid → sequence of patch tokens



- Cut the image into a grid of non-overlapping P×P pixel patches; each patch becomes one token.
- Grid is read row-by-row (raster order) into a 1D sequence — exactly like words in a sentence. Spatial layout is thrown away here; positional encodings (Part II) must put it back.

From patch to embedding: flatten, then project



The text analogy you already know

word	→	token id	→	embedding lookup
patch	→	pixel vec	→	linear projection

Same result: a sequence of D-dim vectors.
The transformer never knows the difference.

- Each 16×16×3 patch is flattened to a 768-vector, then mapped by ONE shared learned matrix E to the model width D. The “visual tokenizer” is a single linear layer learned during training.
- Compare with text: discrete vocabulary + lookup table. Patch pixel values are continuous, so there is no vocabulary — projection replaces lookup. (Implementation note: projection with E is equivalent to a Conv2d with kernel E and stride = P.)

Example: ViT-B(ase)/16 at a glance

“B/16” = Base model, patch size 16.

step	expression	ViT-B/16 @ 224px
input image	$H \times W \times C$	$224 \times 224 \times 3$
number of patches	$N = HW / P^2$	$224 \cdot 224 / 16^2 = 196$
flattened patch	$P^2 \cdot C$	$16 \cdot 16 \cdot 3 = 768$
projection matrix E	$(P^2C) \times D$	768×768
sequence to encoder	$(N + 1) \times D$ (+1 = [CLS])	197×768
attention cost /head/layer	$O((N+1)^2)$	$197^2 \approx 3.9 \times 10^4$ (cf to pixel level tokens!)

- NOTE: 1) for ViT-L(arge)/16, E maps 768 → 1024.
- NOTE: 2) the number of parameters for a ViT is large (multiple attention heads (Q, K, V) for each, MLP layers, depends on D and N etc)

Assembling the input: the [CLS] token

- **Borrowed from BERT: prepend one extra learned vector, x_class**
  - It is not computed from the image — it is a *learned vector*, identical for every input (a set of D numbers found thru' training)
  - Through L layers of *self-attention* this vector can aggregate information from all patches
  - The token's final-layer state z^0_L is used as "the image representation" → classification head
  - Alternative used by many later models: no [CLS]; mean-pool all patch outputs instead
    - Similar performance and is used in several VLM "vision towers" (the network branch that handles images)

$$z_0 = [\ x_class\ ;\ x^1_p\,E\ ;\ x^2_p\,E\ ;\ \ldots\ ;\ x^N_p\,E\ ]\ +\ E_pos$$

$E \in \mathbb{R}^{(P^2C \times D)}$ learned projection $E_pos \in \mathbb{R}^{((N+1) \times D)}$ learned position embeddings

- This is the essentially difference between BERT's input and ViT's input. E_pos covered later in lecture.

Dosovitskiy et al. 2020, Eq. 1; Devlin et al. 2018 (BERT, arXiv:1810.04805) for the [CLS] mechanism.

The complete Vision Transformer

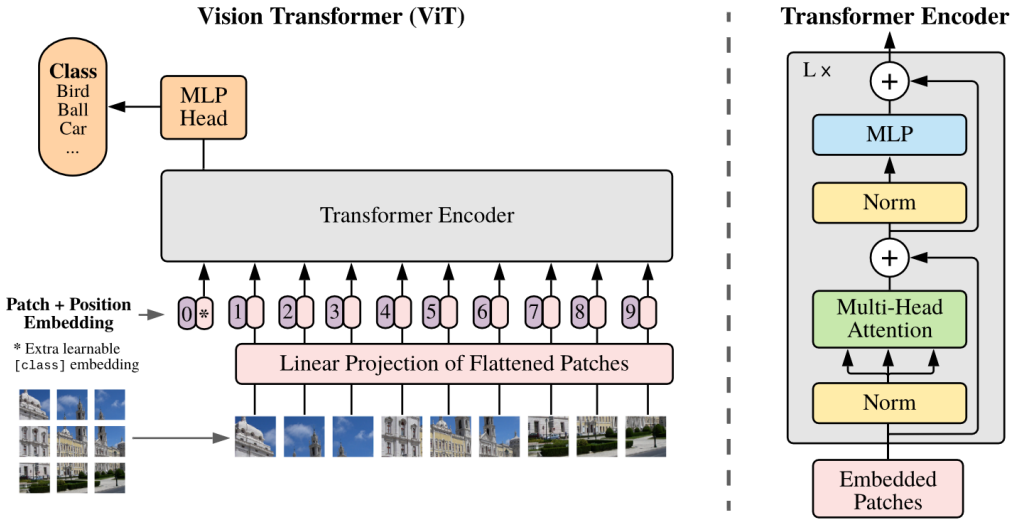


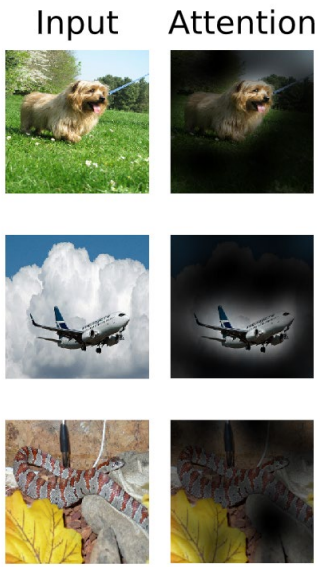
Figure 1: Model overview. We split an image into fixed-size patches, linearly embed each of them, add position embeddings, and feed the resulting sequence of vectors to a standard Transformer encoder. In order to perform classification, we use the standard approach of adding an extra learnable "classification token" to the sequence. The illustration of the Transformer encoder was inspired by Vaswani et al. (2017).

- One encoder block (identical to the text encoder):

$$\mathbf{z}'_\ell = \mathrm{MSA}(\mathrm{LN}(\mathbf{z}_{\ell-1})) + \mathbf{z}_{\ell-1}, \qquad \ell = 1 \ldots L$$
$$\mathbf{z}_\ell = \mathrm{MLP}(\mathrm{LN}(\mathbf{z}'_\ell)) + \mathbf{z}'_\ell, \qquad \ell = 1 \ldots L$$
$$\mathbf{y} = \mathrm{LN}(\mathbf{z}_L^0)$$

- Every patch attends to every patch from layer 1 — global receptive field immediately (a CNN needs many layers to see the whole image)
- Training (original): supervised classification, large datasets (ImageNet-21k: 14M imgs · JFT-300M: 303M), Adam, strong regularization
- What does a ViT "see"?

  Attention map…



NOTE: the core ViT model discussed here is aimed at image classification, not object detection or image segmentation – later ViT variants address this

Credit: Dosovitskiy et al. 2021

Text transformer encoder vs ViT: what changes?

component	text transformer (BERT-style)	Vision Transformer	
tokenization	subword vocabulary (~30–50k discrete ids)	P×P patch grid — continuous, no vocabulary	**CHANGED**
embedding	lookup table (vocab × D)	shared linear projection E ($(P^2C) \times D$)	**CHANGED**
position	1D: sinusoidal or learned	learned 1D over patch index (2D structure emerges — Part II)	**ADAPTED**
[CLS] token	learned summary token	learned summary token	**SAME**
encoder blocks	MSA + MLP, LayerNorm, residuals	MSA + MLP, LayerNorm, residuals	**SAME**
attention	bidirectional, all pairs	bidirectional, all pairs	**SAME**
training	MLM / next-sentence (self-supervised)	supervised classification (originally)	**CHANGED**

IMPORTANT: only the "interface" changed. The computation is untouched — which is exactly why one architecture will later host text and images in the same sequence (VLMs…).

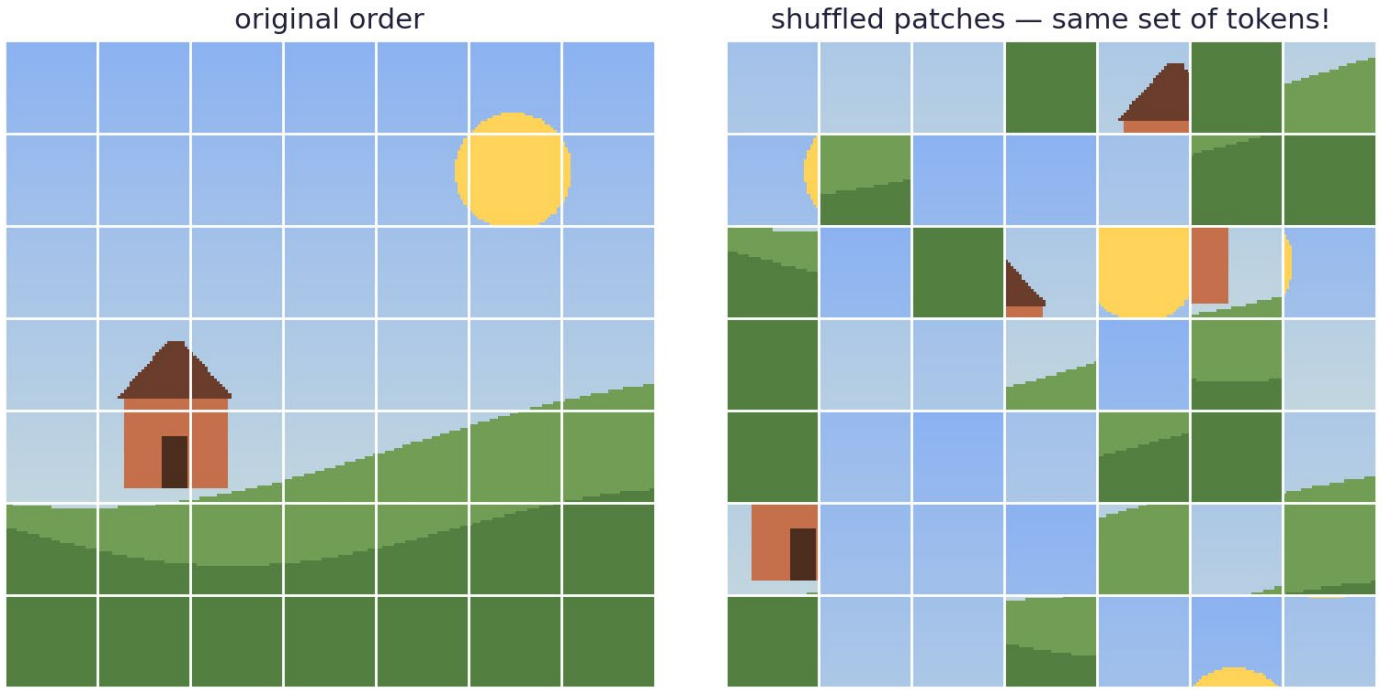
Devlin et al. 2018 (BERT); Dosovitskiy et al. 2020 (ViT).

PART II

Positional encoding for image tokens

Self-attention is permutation-invariant — position must be injected. How, in 2D?

Without positions, ViT sees a bag of patches

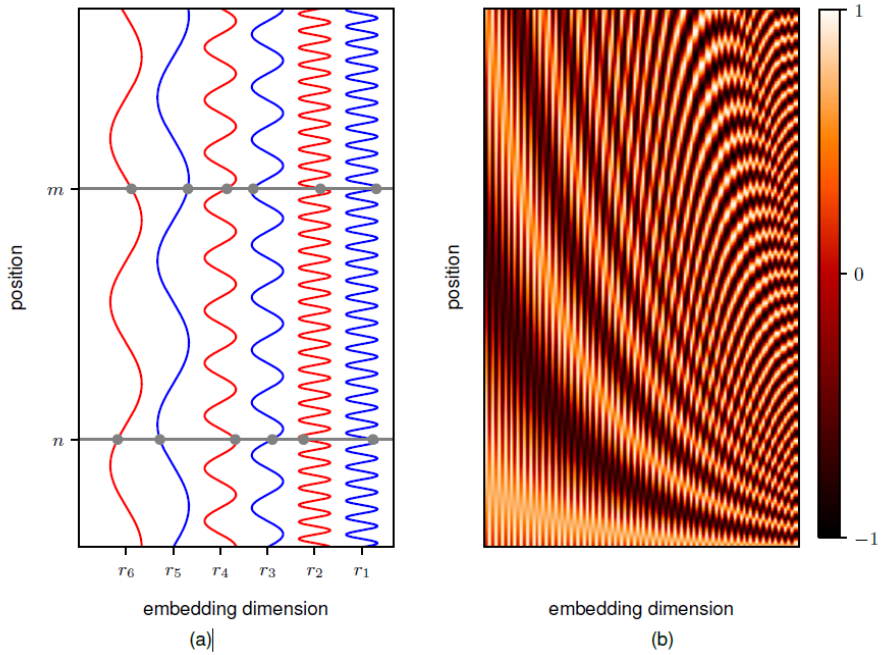


- **Self-attention is permutation-equivariant: reorder the input tokens and the outputs reorder identically — nothing else changes**
  - To a position-free transformer, both images on the left are the same input
  - In text, losing order problematic (“dog bites man”). In vision it destroys all information
    - Pixel spatial arrangement contains all the information, else garbage
    - Patch-level permutations not as bad as pixel level permutation, but still lose accuracy

- Design question: patches live on a 2D grid, but the sequence is 1D. Do we encode “patch #37” — or “row 5, column 2”?

- **Answer: just “patch #37” — and let the model figure the grid out itself.**

Recap from text: sinusoidal encodings




```
PE(pos, 2i)   = sin(pos / 10000^(2i/D))
PE(pos, 2i+1) = cos(pos / 10000^(2i/D))
```

- Fixed (not learned); each position gets a unique multi-frequency fingerprint; relative shifts correspond to linear transforms

- **Three properties made this attractive for text:**
  - extrapolates to unseen lengths (in principle)
  - no parameters to learn
  - Relative position of tokens learned
- ViT tested this method — and found learned embeddings work just as well. Since N is fixed per resolution anyway, ViT simply learns E_pos as free parameters.

Vaswani et al. 2017, §3.5; Dosovitskiy et al. 2020; Figure from C. Bishop, Deep Learning: Foundations and Concepts

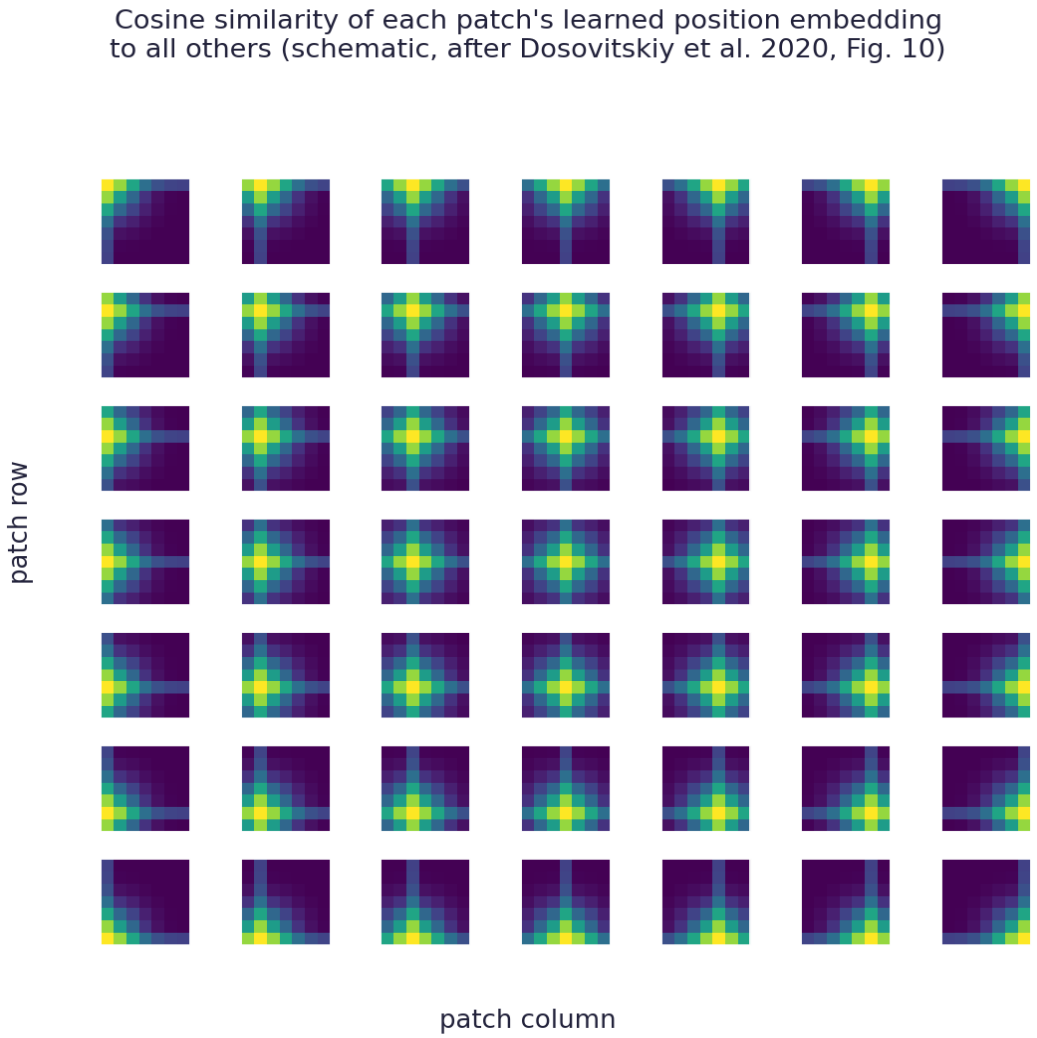
ViT's: learned 1D position embeddings

- **Simple mechanic:**
  - A matrix E_pos of (N+1) rows, one D-vector per sequence slot (incl. [CLS]) — initialized randomly
  - Row i is added to token i's patch embedding (not concatenated — also used in BERT)
  - Trained by backprop like any other weight; N is fixed once resolution and P are fixed
  - "1D" means: the matrix only knows the raster index 0…N — it is told nothing about rows, columns, or that a grid exists
- **Why adding works (intuition):**
  - In high dimension embedding space (like D=768 dims), embeddings and their associated position can occupy "near-orthogonal subspaces"
  - Information is more or less decoupled/disentangled
- **What about 2D nature of positions?**
  - Patch 13 and patch 14 are adjacent in the sequence — but patch 14 and patch 27 are vertical neighbors (in a 14-wide grid). Can the model recover 2D adjacency it was never told about?

→ **next slide: what E_pos actually learned**

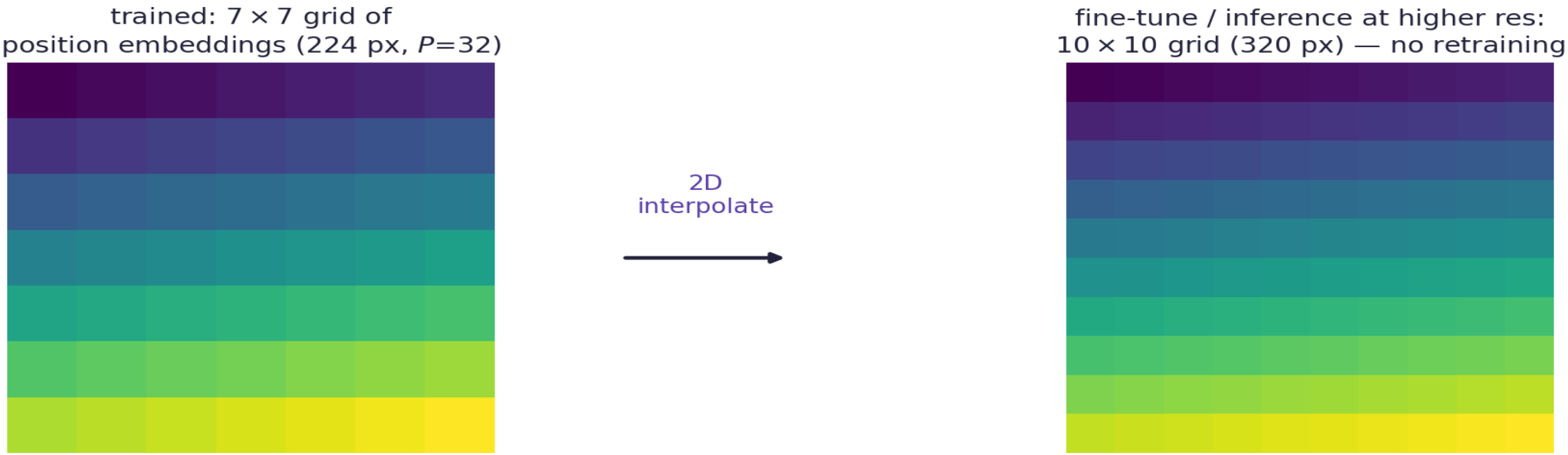
Dosovitskiy et al. 2020, §3.1 & Appendix D.4.

What the embeddings learn: 2D structure emerges



- **How to read this figure:**
  - Outer grid = the 7×7 patch positions. Each cell shows one patch's position embedding compared (cosine similarity) to all 49 — bright = similar
- **Three learned properties with no supervision:**
  - Locality — nearest 2D neighbors have the most similar embeddings
  - Row/column structure — visible bright cross through each cell: same-row and same-column patches are similar
  - Smooth distance decay — similarity falls off gradually with 2D distance
- **The model figured out grid geometry from image statistics**
  - It turns out that more complex hand-designed 2D encodings gain almost nothing - so we just use this simple 1D embedding

Changing resolution: interpolate the embedding grid



- Problem: E_pos has exactly N rows. Feed a larger image → more patches (N) → no embeddings for them (text has the same issue with longer sequences)
- Fix: treat the learned embeddings as samples of a smooth 2D function over the grid; bicubically interpolate to the new grid size.
  - tensor : 7 x 7 x D -> (interpolate to) -> 10 x 10 x D
- Works because of the locality/smoothness the model learned — this is standard trick for fine-tuning ViTs at higher resolution
- Note: interpolation shifts effective patch geometry/layout — short fine-tune at new resolution recovers (and usually improves) accuracy

Dosovitskiy et al. 2020, §3.2; Touvron et al. 2021 (§“FixRes” discussion).

Advanced positional encoding: relative and rotary positions

scheme	core idea	what it buys for vision	seen in
absolute learned (this lecture)	one encoding vector per position added to input embedding	simple; good at fixed resolution; interpolates OK	ViT, CLIP's vision tower
relative bias (not covered)	attention logit gets a learned offset depending on (Δrow, Δcol)	translation-aware; resolution-robust; windowed variants scale to large images	Swin Transformer
2D RoPE (rotary positional embedding) (briefly revisited later)	rotate Q/K by angles ∝ position; relative offsets appear naturally inside QK^T; factorized per axis (x, y — and t for video)	native dynamic resolution — no interpolation step; extends cleanly to time	Qwen-VL family ("M-RoPE"), most current VLM vision towers

- Every pos. encoding scheme answers "how should attention between two patches depend on their spatial relationship?" Absolute answers with where each is; relative/rotary answer with the offset between them — which is what generalizes across resolutions.
- Later we'll see why native dynamic resolution matters: e.g. reading small text in documents.

Liu et al. 2021 (Swin, arXiv:2103.14030); Su et al. 2021 (RoPE, arXiv:2104.09864); Bai et al. 2025 (Qwen2.5-VL, arXiv:2502.13923).

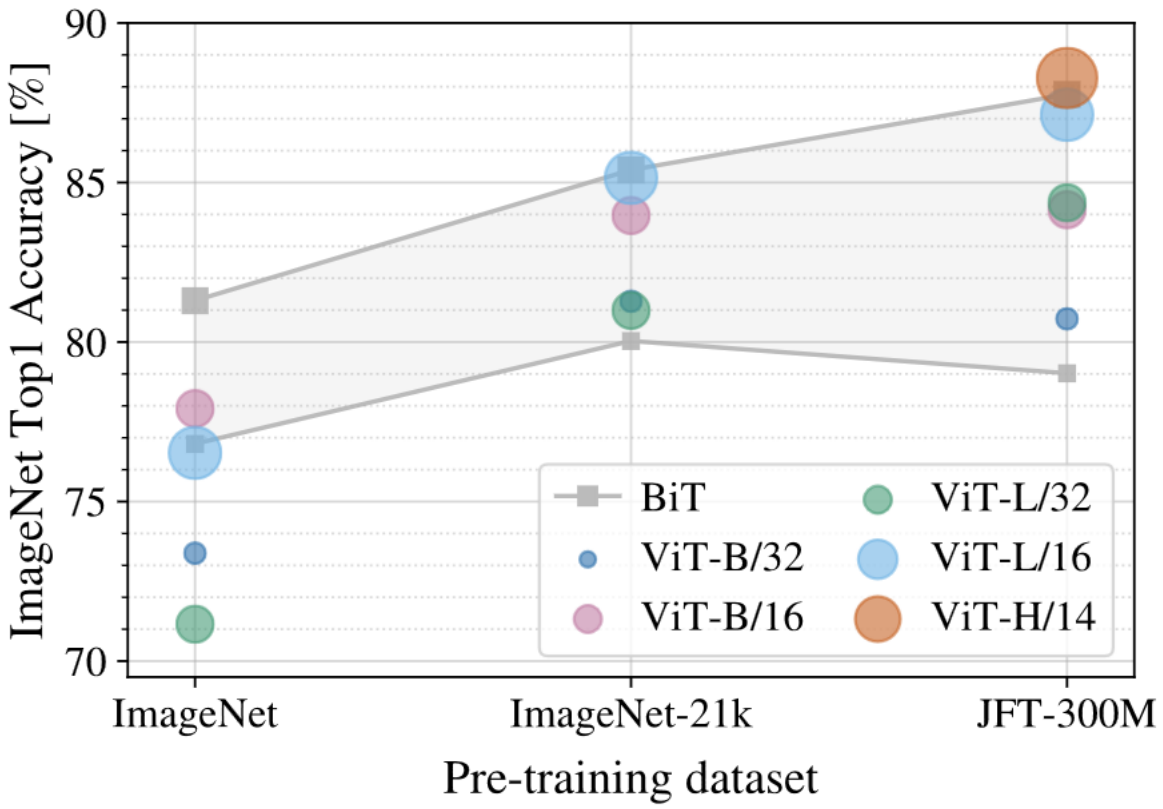
PART III

ViT vs CNN: biases, data, and scale

Inductive biases: what each architecture assumes

	CNN	ViT
locality	baked in: kernels see small neighborhoods; nearby pixels assumed related	none — any patch may attend to any patch from layer 1
translation handling	weight sharing → translation-equivariant features for free	must be learned
scale hierarchy	built in: pooling/stride builds coarse-to-fine pyramid	single resolution throughout (hierarchies later added by Swin-style transformers)
receptive field	grows slowly with depth — global view only near the last layers	global immediately — long-range relations from layer 1

Training data vs model perf - ViT needs scale to win



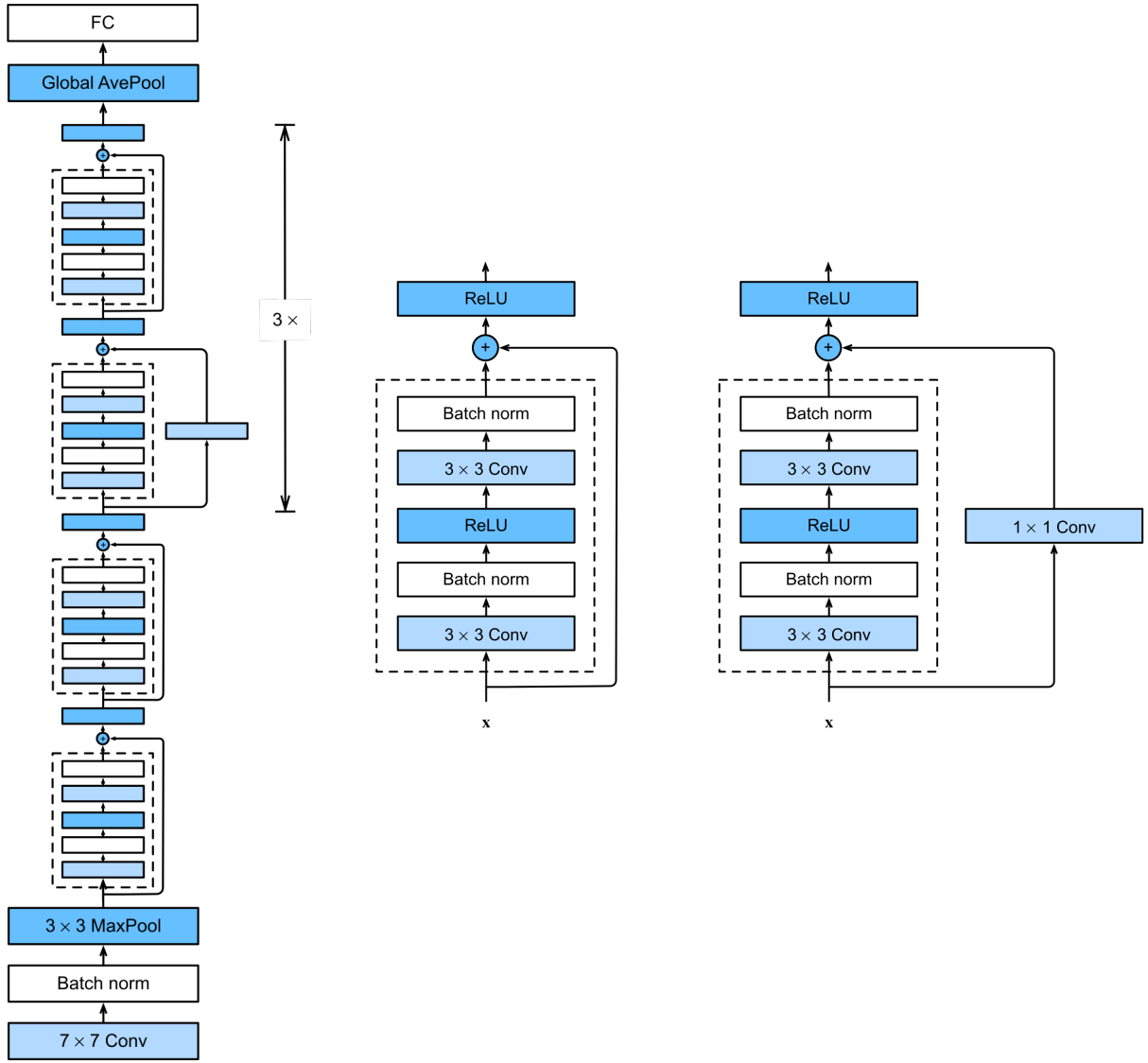
- BiT is a very deep Resnet vision model based on convolutional layers
- Small data (ImageNet-1k only): ResNets win — CNN inductive biases act as regularization
- Large data (JFT-300M): ViT-H(uge)/14 overtakes — learned structure beats built-in structure with very large training data
- And ViT is 2–4× cheaper to pre-train at matched accuracy

- **Thus: modality-agnostic transformers + huge web-scale image–text pairs can ensure useful pretraining for language/vision tasks → CLIP**

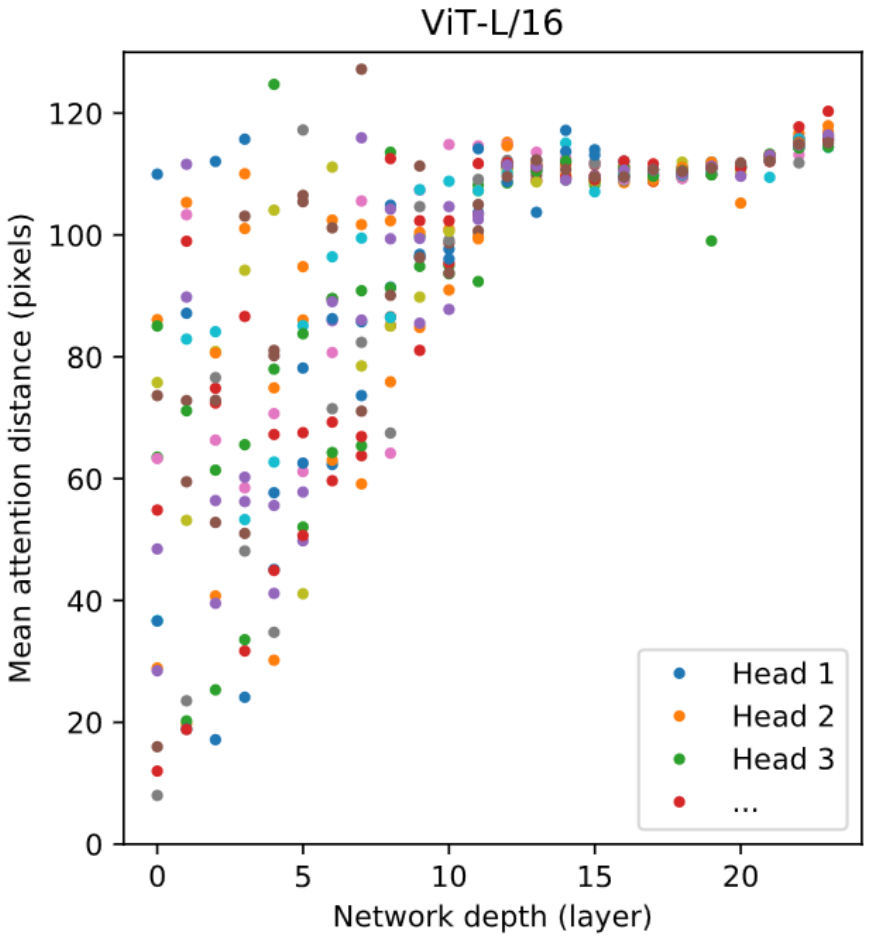
Dosovitskiy et al. 2020.

Interlude: ResNET architecture

- ResNeT = Residual Network
- ML architecture that composed (convolution-based) residual blocks
- Residual block evaluate x' = x + F(x) where F(x) is the output of the convolution block (with Batch/layer norm etc – see figure)
- Why? Stacking residual blocks allows exceptionally deep CNN's with great expressive power to train stably
- Why? The residual block skip connection allows one to learn an identity map (input propagates to output, unchanged) – we only use capacity if needed, else learn identity map, helps avoid gradient vanishing as passes through such blocks (errors compound with depth but we always ADD some gradient back due to skip "identity")



Inside a trained ViT: “local heads” (CNN-like) emerge



- Mean attention distance = how far (in pixels) a head looks on average — a “receptive field size” for attention
- Early layers: heads SPLIT — some behave like small conv kernels, others are already global
- Deep layers: everything goes global
- **Interpretation: locality is useful, so ViT learns it where needed — but is never limited by it. The CNN inductive bias is a special case ViT can discover.**

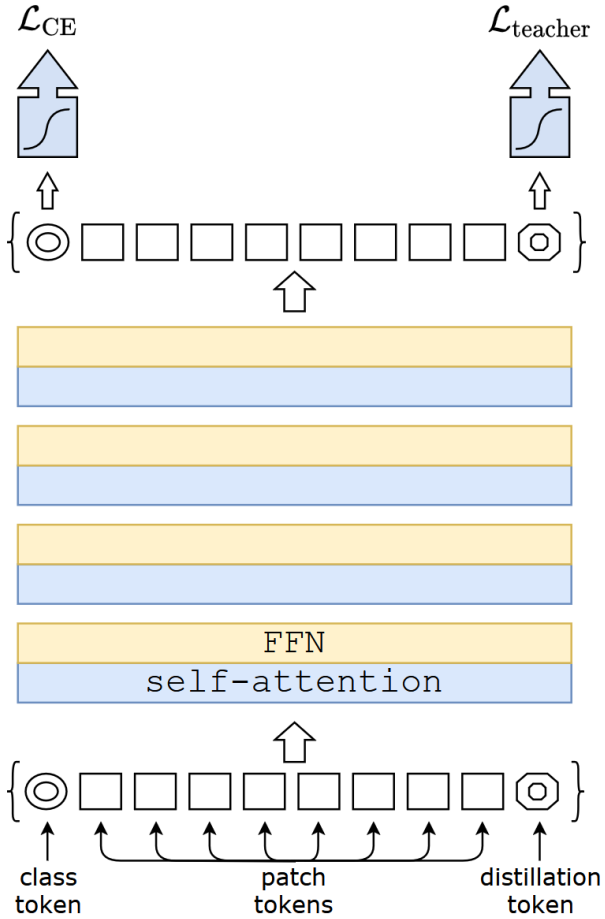
ViT models - the impact of patch-size

model	layers	width D	heads	params	tokens @224 (P=16)
ViT-Base/16	12	768	12	86 M	196 + 1
ViT-Large/16	24	1024	16	307 M	196 + 1
ViT-Huge/14	32	1280	16	632 M	256 + 1 (P=14)

- **Patch size P is a resolution/compute dial, independent of model size:**
  - P=32 → 49 tokens: cheap, coarse — fine detail lost inside patches
  - P=16 → 196 tokens: the standard operating point
  - P=8 → 784 tokens: fine-grained, 16× the attention cost of P=32 — token count scales as $1/P^2$
- **Note: to interpret models/papers: "ViT-L/14 @ 336px" = Large model, patch size 14 pixel, 336-pixel input → $(336/14)^2 = 576$ patch tokens — this exact configuration is CLIP's image tower (covered tomorrow)**

DeiT: reducing the data gap with distillation

- **Goal: train a competitive ViT on ImageNet-1k only (1.3M images), on one machine, in ~3 days (vs original ViT on 300M images….)**
- **Step 1 — aggressive augmentation and regularization:**
  - RandAugment, Mixup, CutMix, random erasing, stochastic depth — synthetic data diversity substitutes for massive unique data set
- **Step 2 — learn the missing inductive bias from a CNN teacher:**
  - include a distillation token ([DIST]) alongside [CLS] to input patch embeddings; it is trained to predict a pretrained CNN's (hard) labels (the CNN is the teacher model)
  - Use a ground truth cross-entropy loss (CLS prediction) plus a 'distillation loss' (teacher vs ViT student)
  - The CNN's built-in locality/translation knowledge transfers into the ViT via optimization of this loss function
- **Result: DeiT-B ≈ 83.4% ImageNet top-1 with no external data — training ViTs became practical for many more**
- Observation: the "data hunger" of ViT can be worked around – here using augmentation + teacher knowledge instead of more images
- BUT in some domains (like medicine) augmentation may be a problem



Touvron et al. 2021, "Training data-efficient image transformers…" (DeiT, arXiv:2012.12877).

ViT Summary

Tokens

Patches are the words of vision: P×P crop → flatten → ONE shared linear projection. 224px, P=16 → 196 tokens + [CLS]. The transformer itself is untouched.

Position

Self-attention is permutation-invariant — position must be injected. ViT: learned 1D embeddings, which discover 2D grid structure on their own; interpolate them to change resolution; modern towers use factorized 2D RoPE.

Biases & scale

ViT trades CNN's built-in locality/translation biases for data. Small data → CNN wins; web scale → ViT wins (and re-learns locality where useful). DeiT shows the gap can be closed with augmentation + model distillation.

ViT – where we see it later

Lecture 2	**CLIP**	A ViT is the image encoder; its [CLS]/pooled output is matched against text embeddings using contrastively loss
Lecture 3	**Flamingo · BLIP-2 · LLaVA**	A (frozen) ViT produces the patch tokens that are matched to LLM embeddings
Lecture 4	**Production VLMs**	ViT “towers” with 2D-RoPE, native resolution – real-world application
Lecture 5	**Video models**	Patches + a time axis → spatiotemporal “tubelets”; “factorized space-time attention”
Lecture 7	**Diffusion Transformers (DiT)**	Even image generators are ViTs — denoising operates on patch tokens of a latent image

References and Resources

- **Primary sources for this lecture (for those who want to read on – these are all research papers so very technical)**
  - Dosovitskiy et al. (2020). An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale. ICLR 2021. arXiv:2010.11929
  - Touvron et al. (2021). Training data-efficient image transformers & distillation through attention (DeiT). ICML 2021. arXiv:2012.12877
- **Background assumed (basic detail/intuitions etc)**
  - Vaswani et al. (2017). Attention Is All You Need. arXiv:1706.03762 · Devlin et al. (2018). BERT. arXiv:1810.04805
- **Mentioned in passing (out of scope):**
  - Liu et al. (2021). Swin Transformer. arXiv:2103.14030 · Su et al. (2021). RoFormer/RoPE. arXiv:2104.09864 · Bai et al. (2025). Qwen2.5-VL. arXiv:2502.13923
- **Online to try:**
  - Hugging Face Computer Vision Course — “Vision Transformers” unit: huggingface.co/learn/computer-vision-course
  - Run a pretrained ViT in a few lines — https://huggingface.co/google/vit-base-patch16-224